Discrete Fourier Transform

Zhuoqi Cheng
zch@mmmi.sdu.dk
SDU Robotics

SDU

Topics to be covered in this course

→ Sampling and reconstruction
→ Aliasing
→ Quantization and dynamic range
→ Implementation
→ Conversion time-frequency domain
→ Z transform
→ Linear Time Invariant system (LTI)
→ System analysis
→ Window functions
→ Filter design
→ Impulse response (FIR and IIR)

Reference book

Chapter 4. Discrete Fourier Transform and Signal Spectrum
Page 87-136

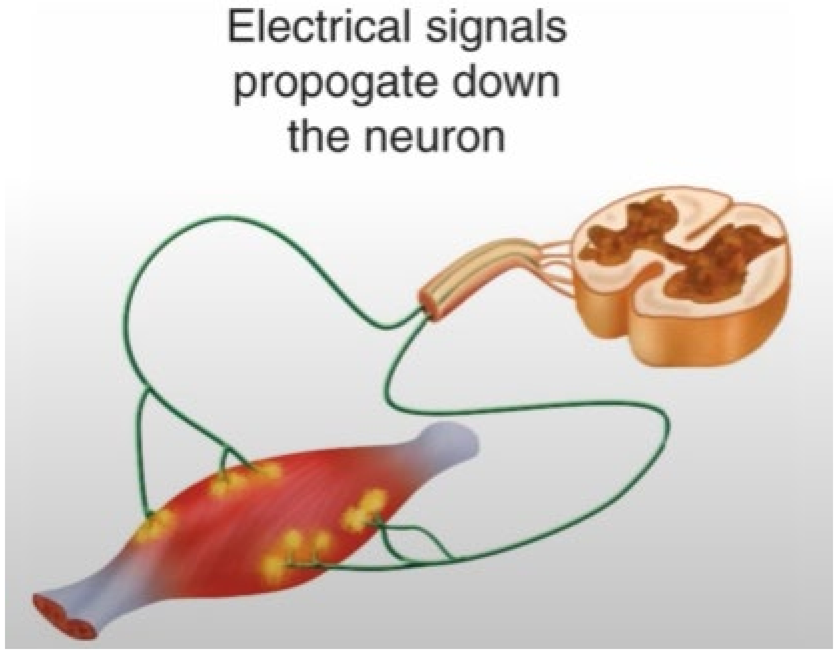
CHAPTER

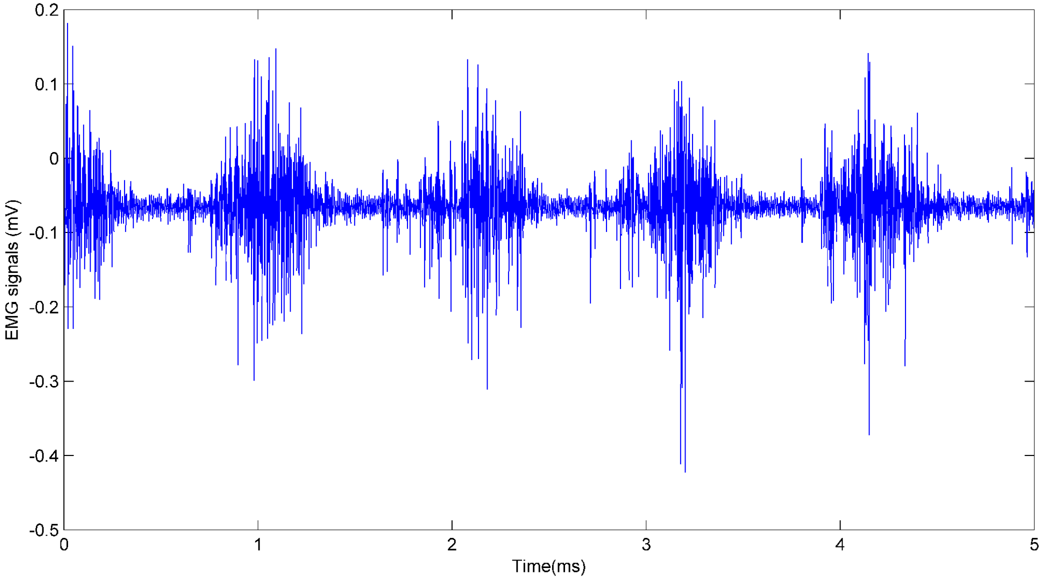
4

Discrete Fourier Transform and Signal Spectrum

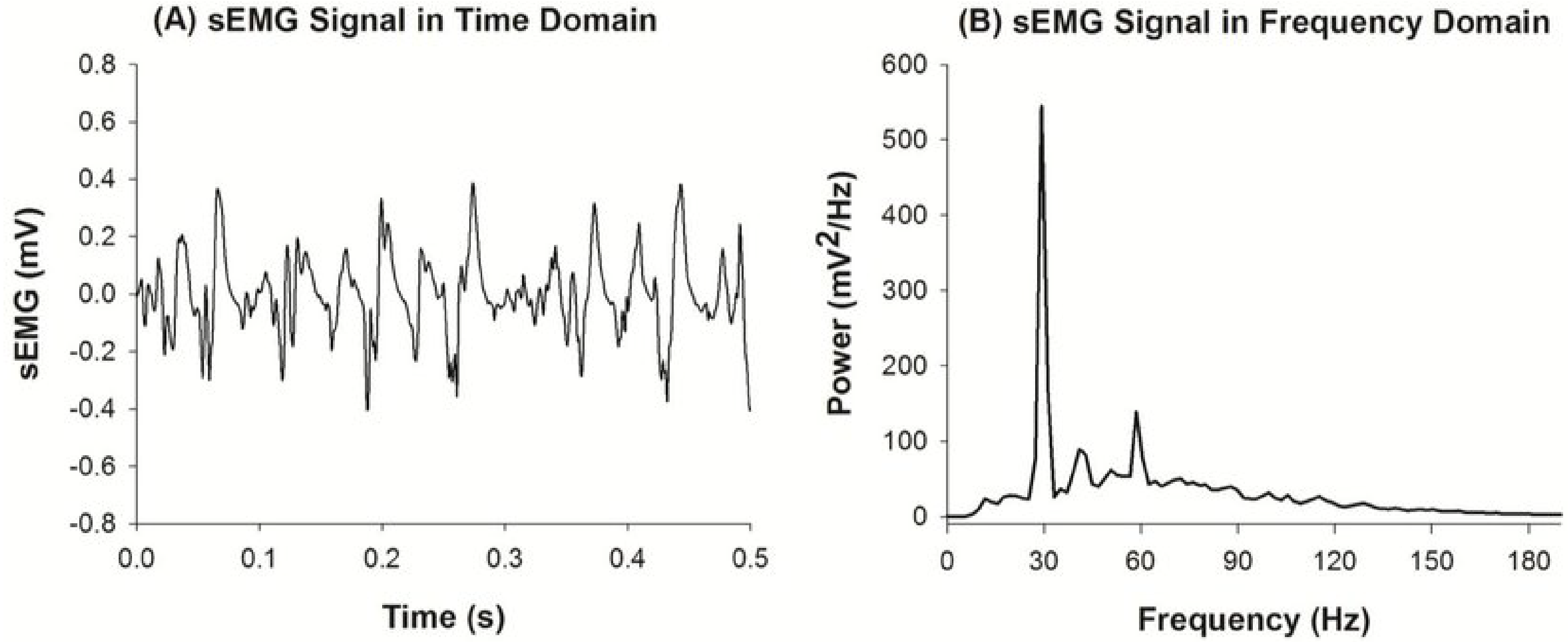
SDU

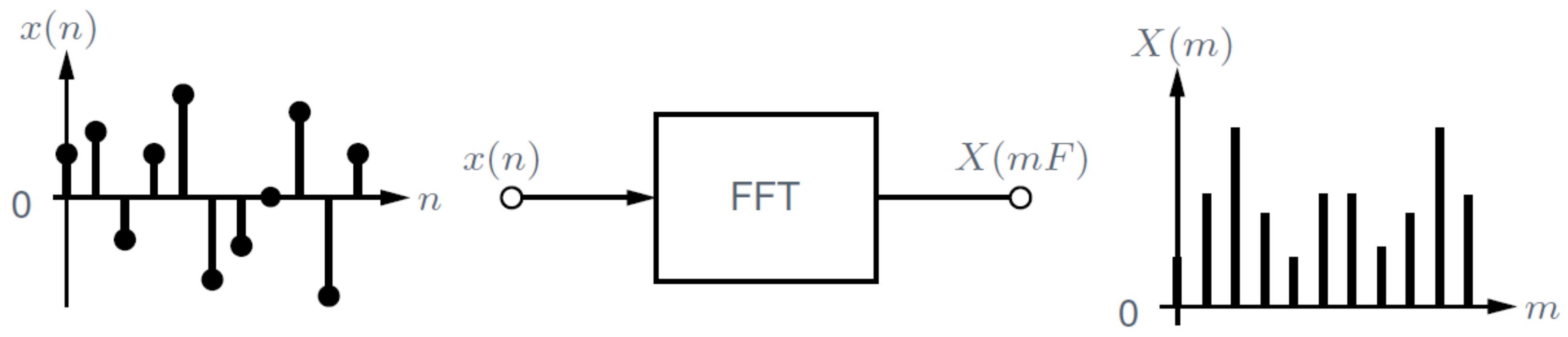
Muscle activity by Electromyography (EMG)



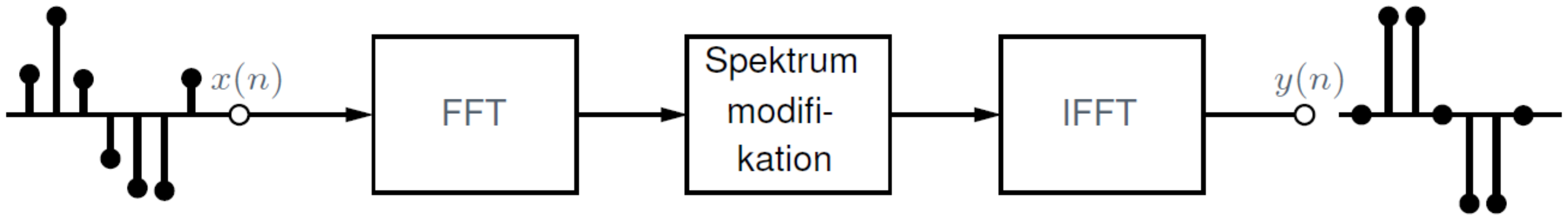


Convert time domain signal to frequency domain for analysis





x(n)
0
n
x(n)
FFT
X(mF)
X(m)
0
m

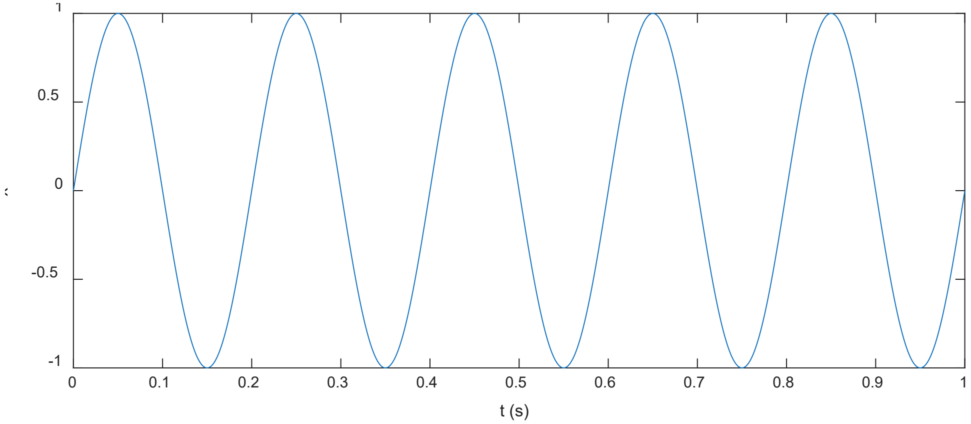


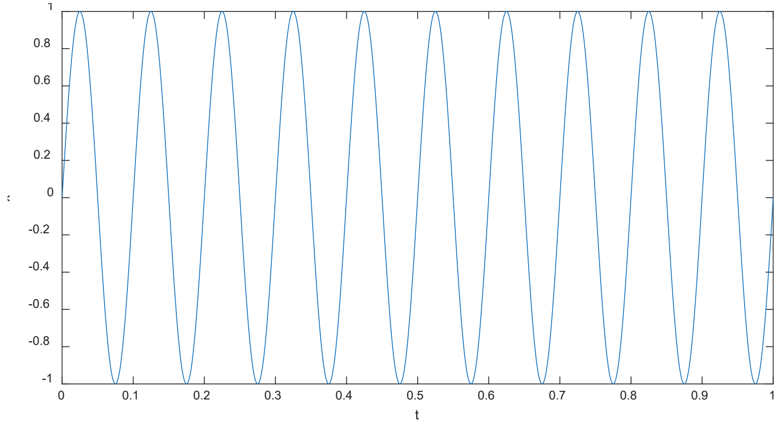
x(n)
FFT
Spektrum modifi- kation
IFFT
y(n)

Fourier Transform – analog signal

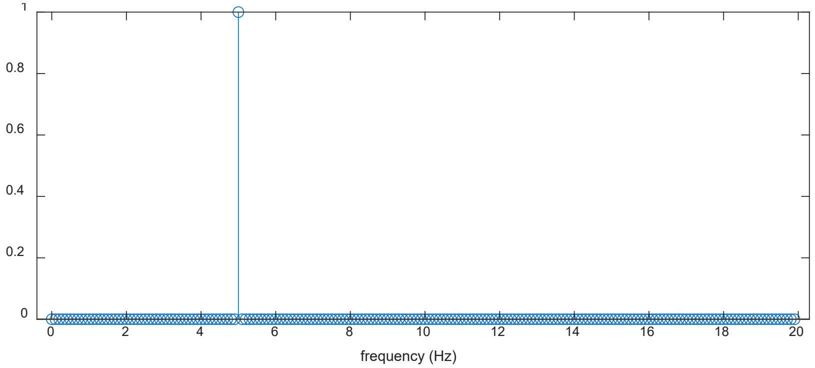
$$x(t) = \sin(2\pi f\ t)$$

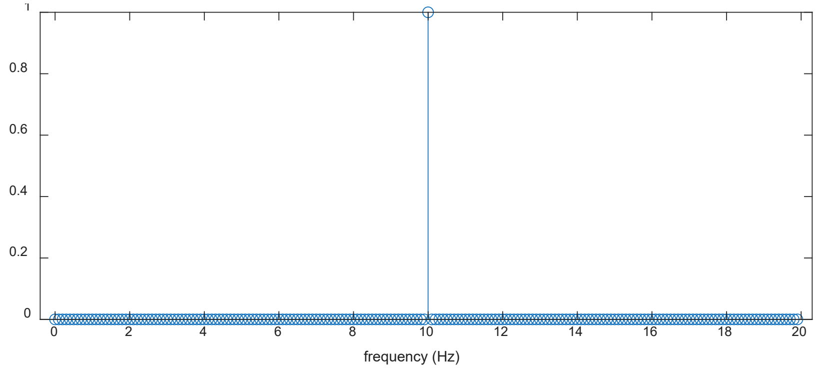
Time domain:





Frequency domain:





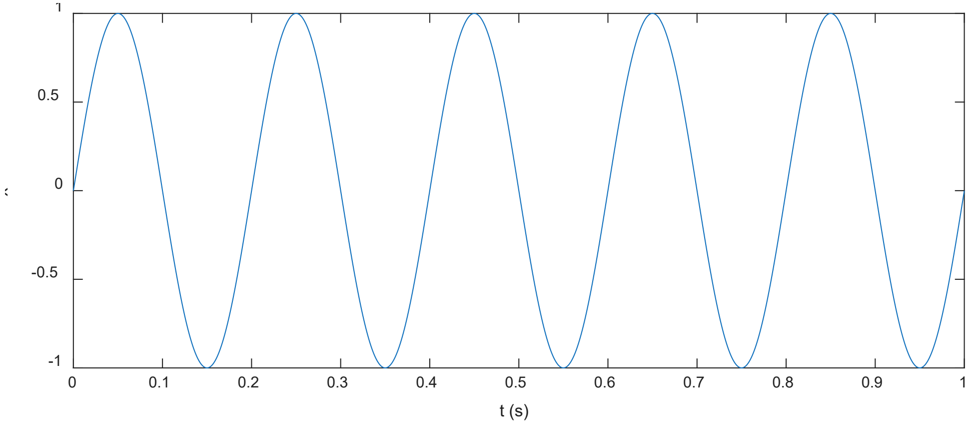
SDU

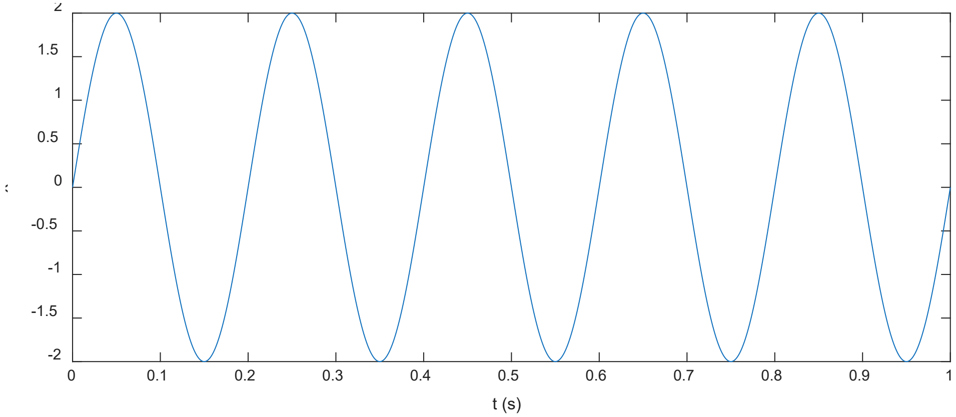
Fourier Transform – analog signal

$$x(t) = \mathrm{A}\sin(2\pi f\ t)$$

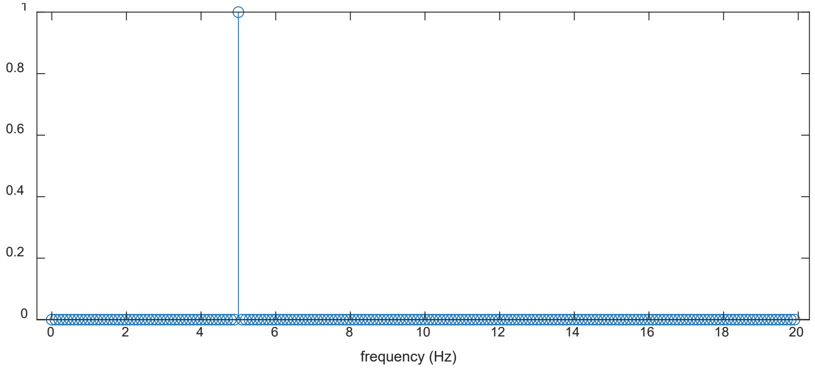
When A = 2

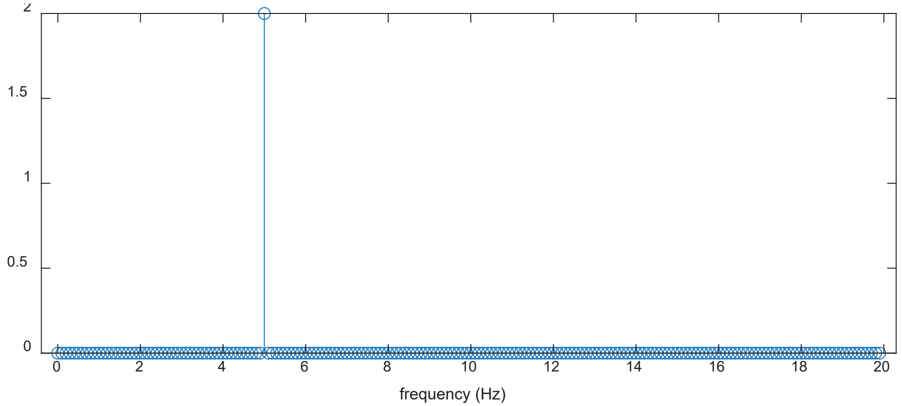
Time domain:



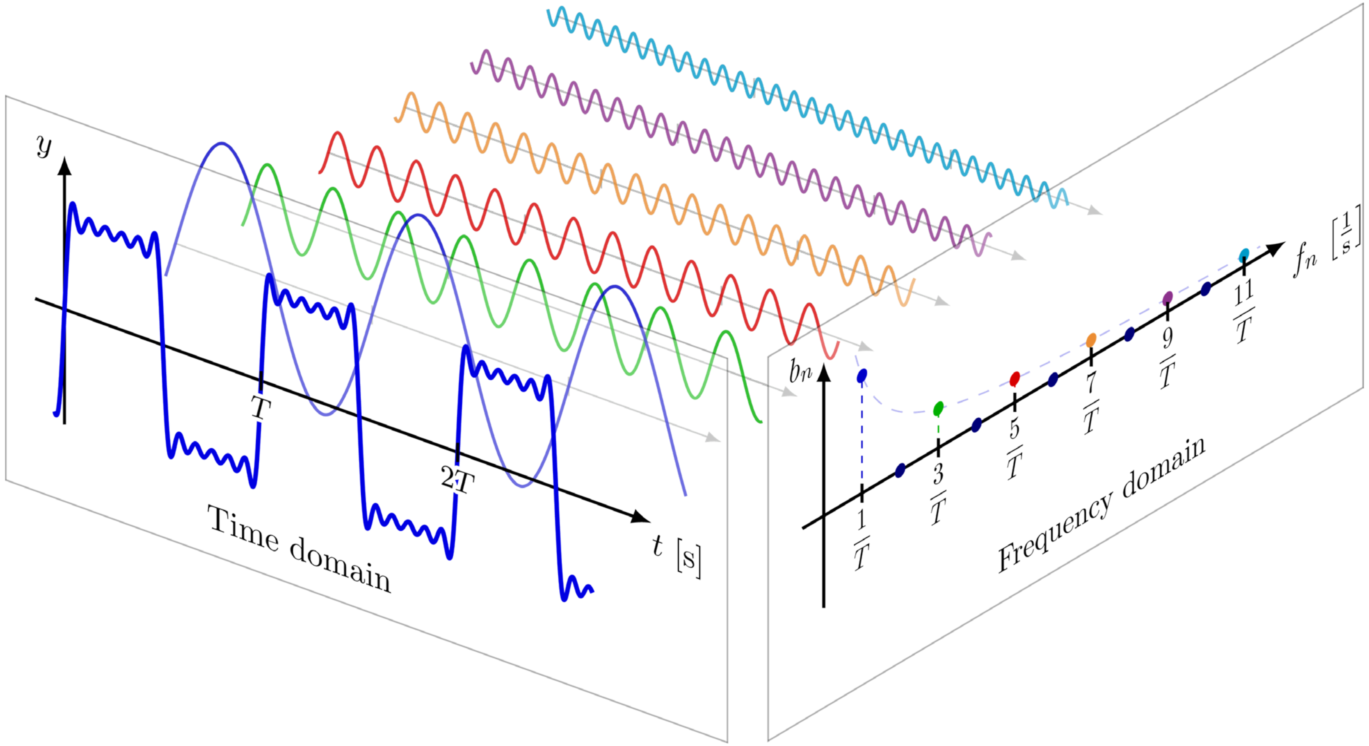


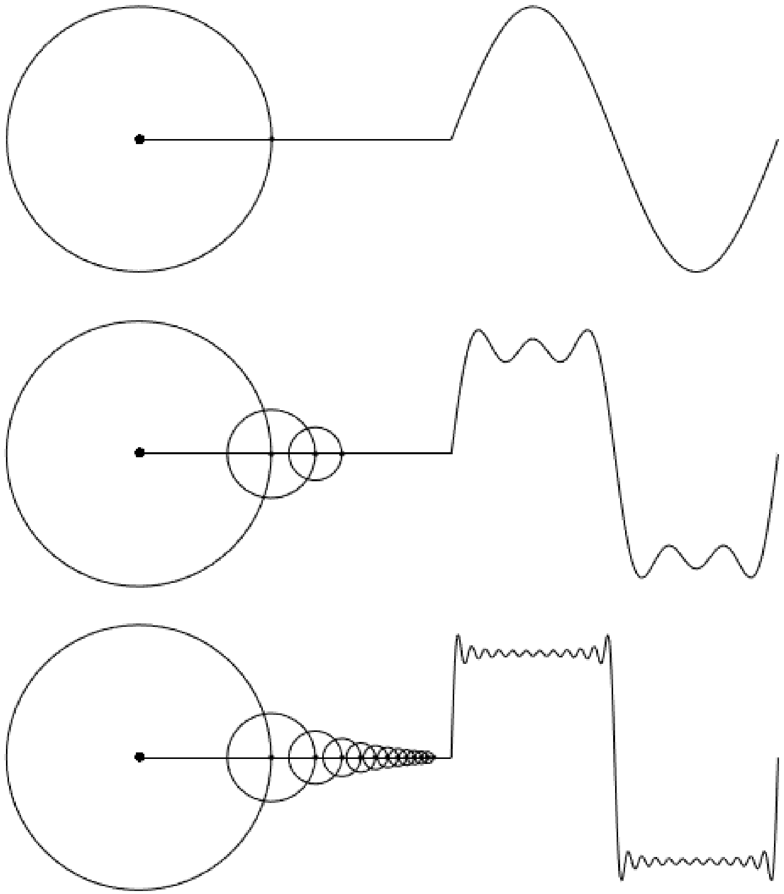
Frequency domain:





Time – Frequency convert





SDU

Fourier Transform – analog signal

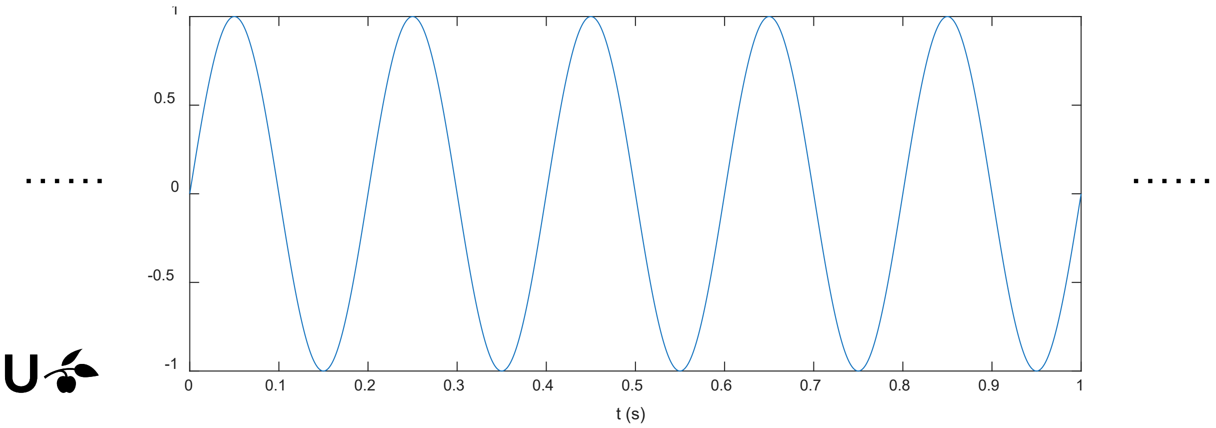
The Fourier transform is a mathematical function that provides frequency spectral analysis.

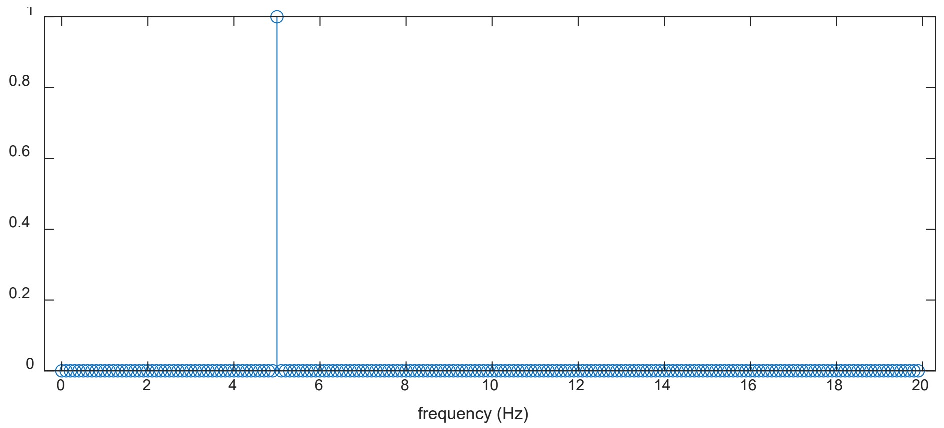
Given a signal $x(t)$, its **Fourier Transform** is defined as

$$X(\omega) = \mathcal{F}\big(x(t)\big) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t}\,dt \qquad \boxed{\omega = 2\pi f}$$

The signal $x(t)$ can also be found from the spectrum $X(\omega)$ via **Inverse Fourier Transform**

$$x(t) = \mathcal{F}^{-1}\{X(\omega)\} = \frac{1}{2\pi}\int_{-\infty}^{\infty} X(\omega)e^{j\omega t}\,d\omega$$





Discrete Fourier Transform

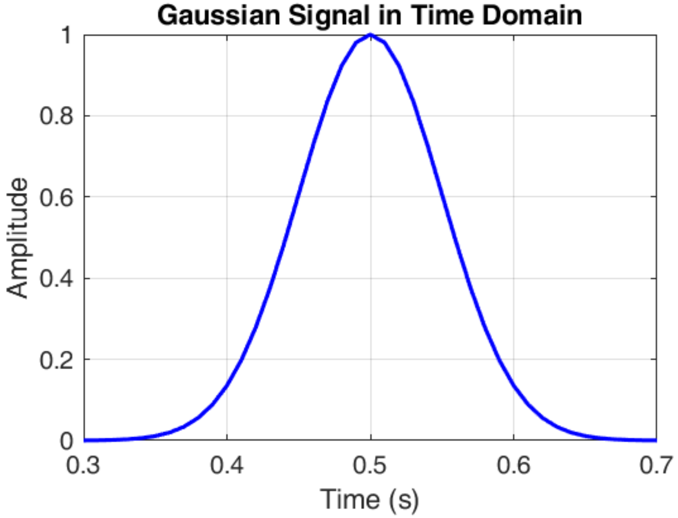
$$X(f) = \mathcal{F}\big(x(t)\big) = \int_{-\infty}^{\infty} x(t)e^{-j2\pi ft}\,dt$$

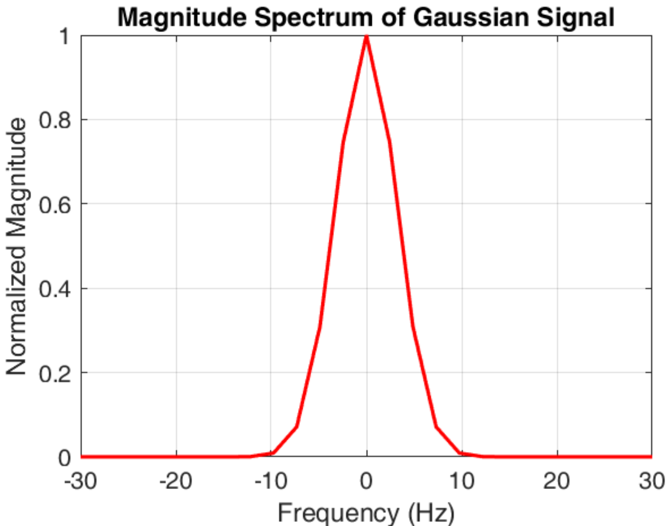
We must calculate the integral over time t from $-\infty$ to ∞

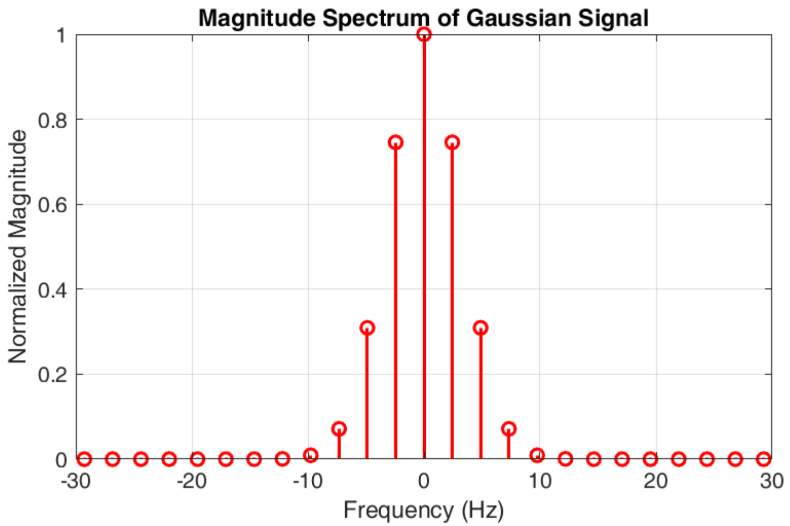
For digital signal, we can calculate the **spectrum** function $X(f)$ by discretizing the frequency range:

$$X(mF) = \int_{-\infty}^{\infty} x(t)e^{-j2\pi mFt}\,dt$$

$$\boxed{f = mF}$$







$$X(mF) = \int_{-\infty}^{\infty} x(t)e^{-j2\pi mFt}\,dt$$

Given the sequence $x(nT)$ which has N samples,

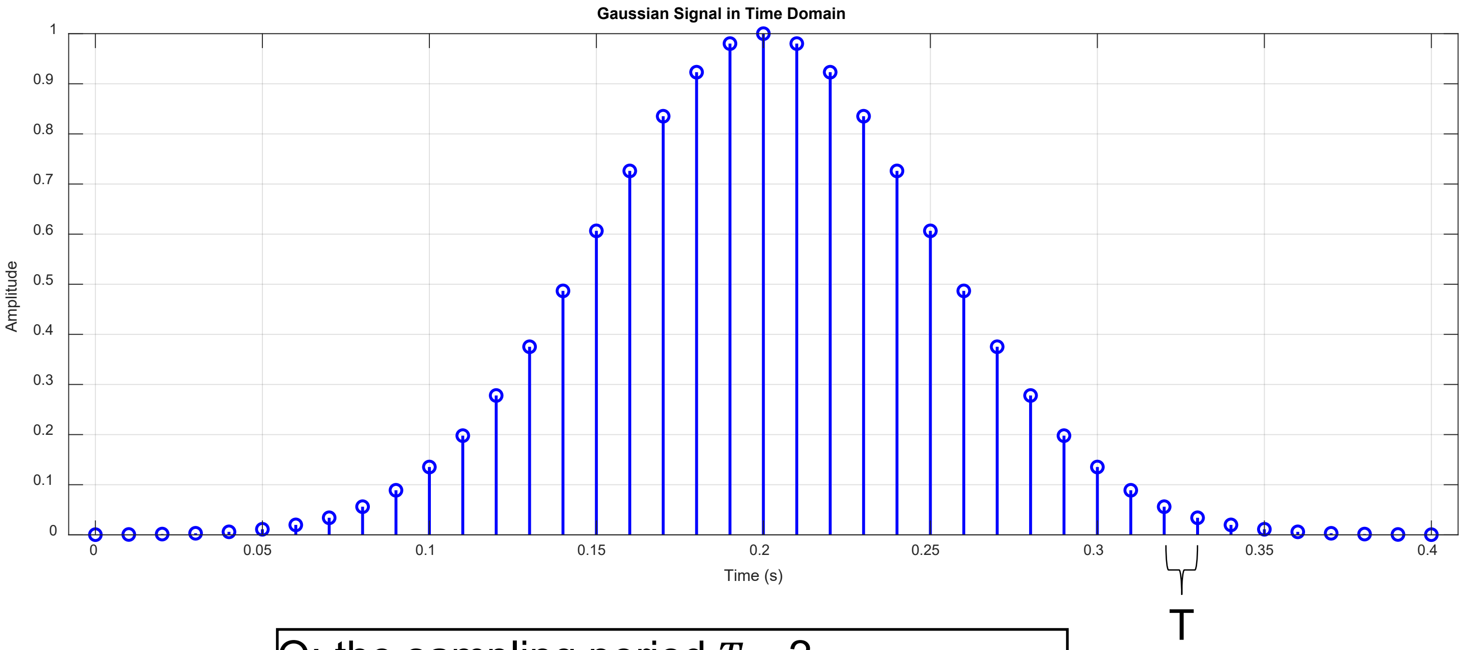
$$X(mF) = T\sum_{n=0}^{N-1} x(nT)e^{-j2\pi mFnT}$$

Given $f_s = NF$
$FT = 1/N$

The final DFT can be written as

$$X(m) := \sum_{n=0}^{N-1} x(n)e^{-j\frac{2\pi mn}{N}}$$

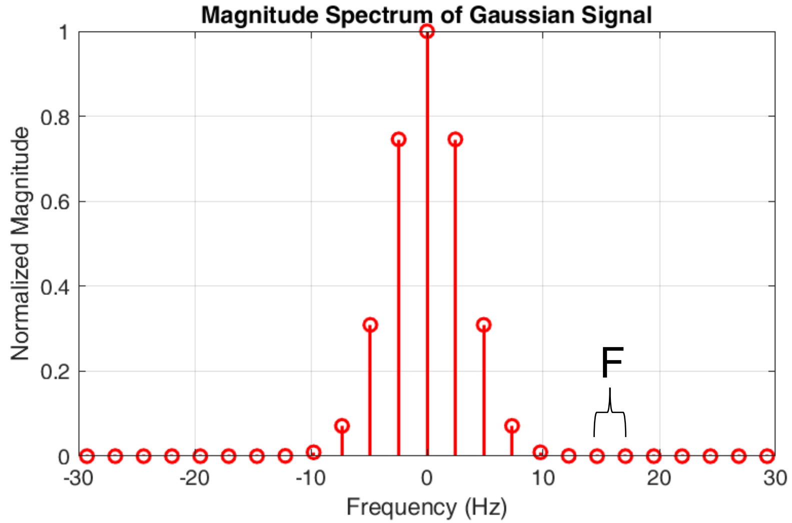
The frequency spacing between the consecutive spectral lines, called the frequency resolution, F. It also represents the m^{th} harmonic.



Q: the sampling period T =?

Then the sampling frequency $f_s = \frac{1}{T}$ =?

In this example, N=41



SDU

Inverse discrete Fourier transformation

Inverse Discrete Fourier Transform (IDFT) is used to calculate the sequence $x(n)$ giving the spectrum function $X(f)$.
For n = 0, 1, . . . , N−1

$$x(n) = \frac{1}{N} \sum_{m=0}^{N-1} X(m)\, e^{j2\pi \frac{mn}{N}}$$

It should be noted that both $X(f)$ and $x(n)$ are periodic, i.e.

$$x(n) = x(n + kN)$$
$$X(m) = X(m + kN)$$

Because $x(n)$ is a complex number, it can be rewritten as

$$x(n) = \frac{1}{N}\left(\sum_{n=0}^{N-1} X(m) \cos\left(\frac{2\pi mn}{N}\right) + j \sum_{n=0}^{N-1} X(m) \sin\left(\frac{2\pi mn}{N}\right)\right)$$

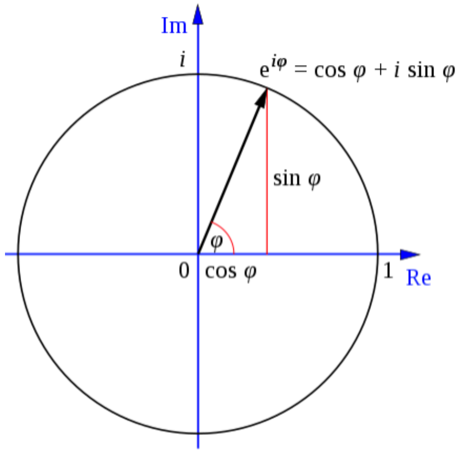
Exponential sequence

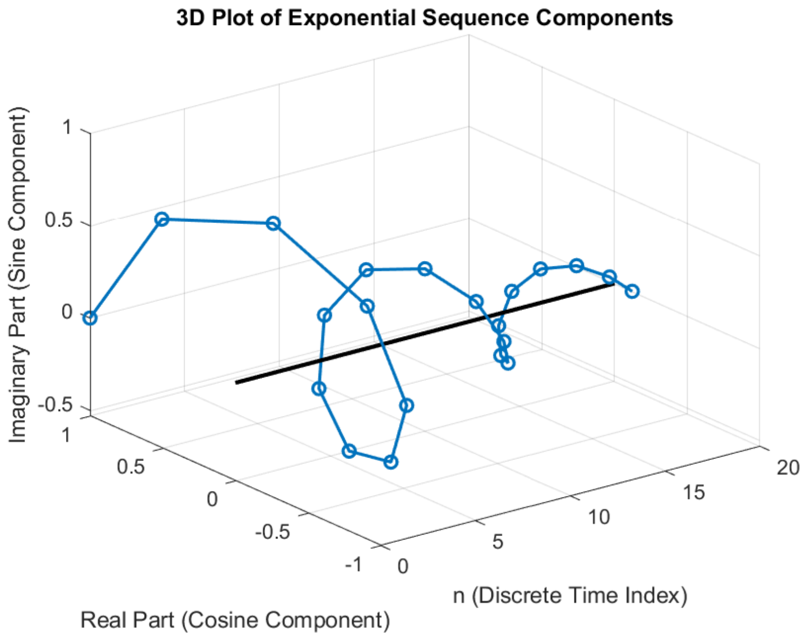
$$x(n) = A\alpha^n$$

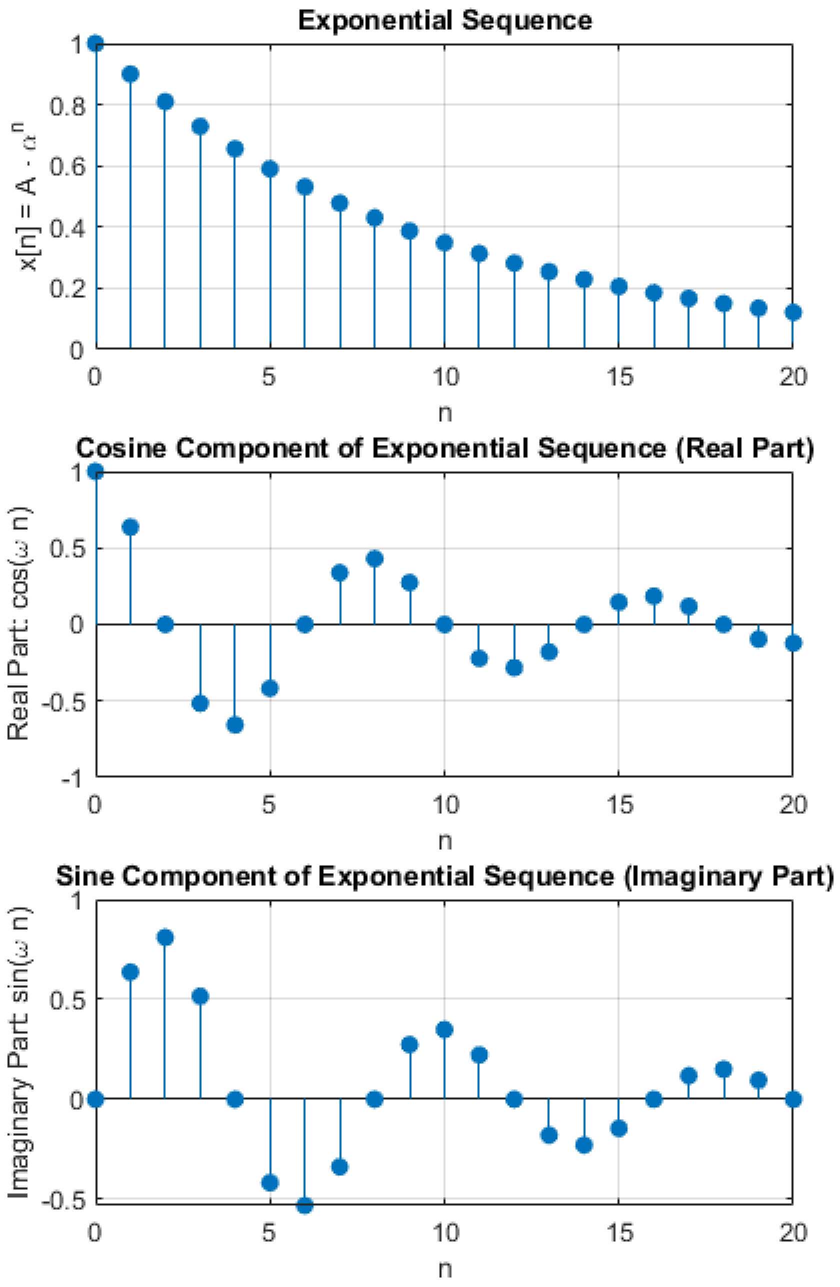
→ Draw an example $x(n) = 0.9^n,\ n \geq 0$

→ Complex exponential $x(n) = e^{j\omega_0 n} = \cos(\omega_0 n) + j\,\sin(\omega_0 n)$

Question:
$\omega_0 = ?$







SDU

Summary

For N-point DFT, the following transformation can be used for calculation:

$x(n)$ is a sequence sampled with interval T
The N-points DFT of $x(n)$ is given as

$$X(m) = \sum_{n=0}^{N-1} x(n) W_N^{mn}$$

for m = 0, 1, …, N-1
Where $W_N = e^{-j2\pi/N}$

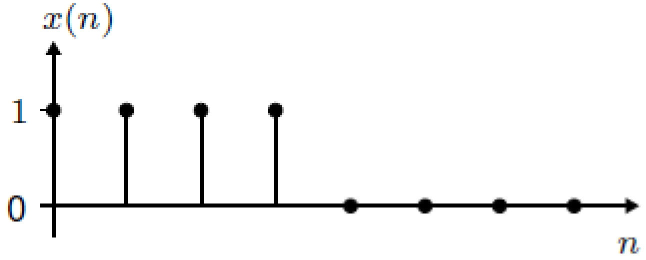
The sequence $x(n)$ can be found from the spectrum $X(m)$ as

$$x(n) = \frac{1}{N} \sum_{m=0}^{N-1} X(m) W_N^{-mn}$$

for n = 0, 1, …, N-1

Example - DFT

Considering the following square sequence $x(n)$



Its spectrum function can be calculated as follows

$$X(m) = \sum_{n=0}^{N-1} x(n) W_N^{mn}$$

Where $W_N = e^{-j2\pi/N}$

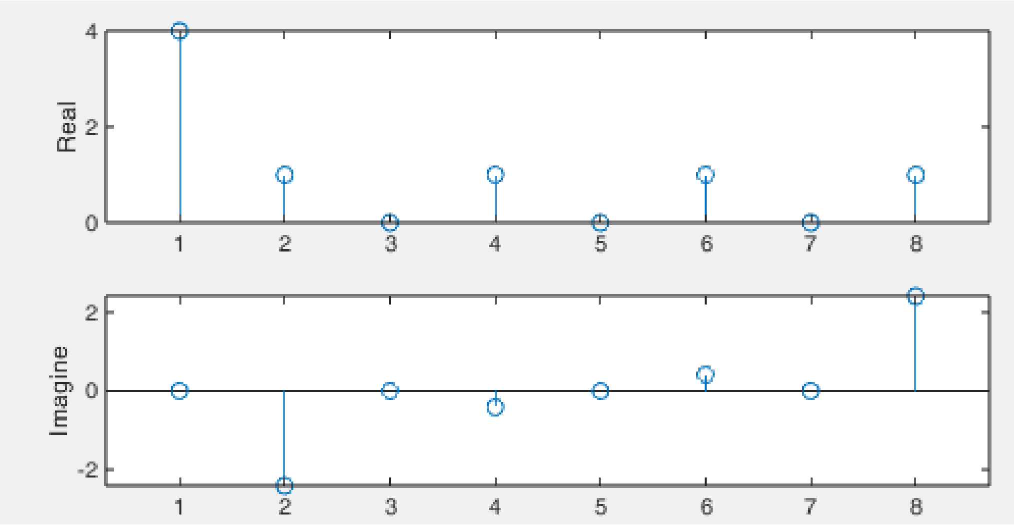
Since N=8, we have

$$X(m) = \sum_{n=0}^{7} x(n) W_8^{mn}$$
$$W_8 = e^{-j2\pi/8}$$

Thus, the spectrum function becomes

$$X(m) = \sum_{n=0}^{7} x(n)\left(\cos\left(\frac{mn\pi}{4}\right) - \mathrm{j}\sin\left(\frac{mn\pi}{4}\right)\right)$$

for m = 0, 1, … , 7




```
%% run Matlab the following

x = [1 1 1 1 0 0 0 0]; % input signal
X = zeros(1,8); % initialize

for m = 0:7
for n = 0:7
X(m+1) = X(m+1) + x(n+1)*(cos(m*n*pi/4) -
i*sin(m*n*pi/4));
end
end
figure();
subplot(2,1,1)
stem(real(X))
subplot(2,1,2)
stem(imag(X))
```

```
% try Matlab inherent function
Y= fft(x);
% plot Y and check the results
```

Example – IDFT

Considering the following sequence in frequency domain

$$\mathrm{X}(m) = [10, \quad 2+j, \quad 0, \quad 2-j]$$

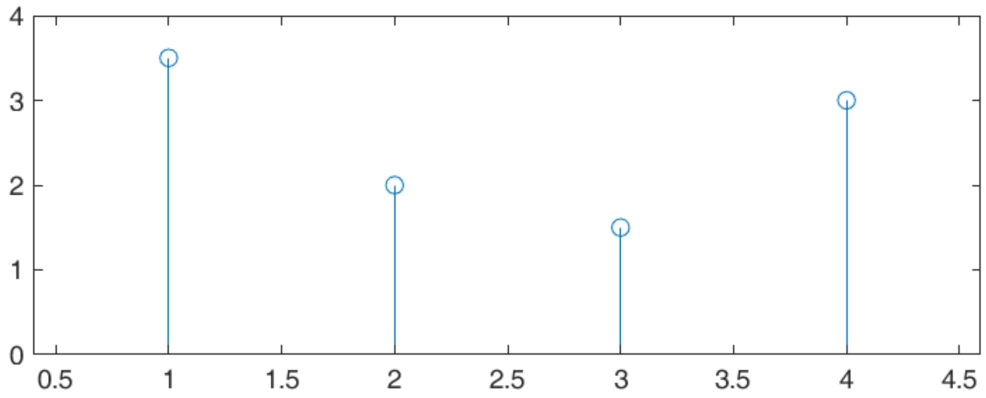
The iDFT equation is

$$x(n) = \frac{1}{N}\sum_{m=0}^{N-1} X(m)\, e^{j2\pi mn/N}$$

Calculate the Time-domain signal:
N = ?

```
%%
X = [10 2+i 0 2-i]; % input signal
x = zeros(1,4);

for n = 0:3
for m = 0:3
x(n+1) = x(n+1) + X(m+1)*exp(i*2*pi*m*n/4);
end
end
x = x / 4;
stem(x)
```




```
% again, try Matlab inherent function
xx = ifft(X);
% plot xx and check the results
```

SDU

Amplitude & phase & power spectrum

The amplitude and phase of $X(m)$ is

$$\mathrm{A_m} = \frac{1}{\mathrm{N}}|X(m)| = \frac{1}{N}\sqrt{\left[Re\left(X(m)\right)\right]^2 + \left[Im\left(X(m)\right)\right]^2} \quad \text{and} \quad \angle X(m) = \tan^{-1}\left(\frac{Im\left(X(m)\right)}{Re\left(X(m)\right)}\right)$$

Where $k = 0, 1, 2, \ldots, N-1$

Power spectrum is defined:

$$P(m) = \frac{1}{\mathrm{N}^2}|X(m)|^2 = \frac{1}{N^2}\left(\left[Re\left(X(m)\right)\right]^2 + \left[Im\left(X(m)\right)\right]^2\right)$$

What we talking about here is in the frequency domain!

Fast Fourier Transform

Zhuoqi Cheng
zch@mmmi.sdu.dk
SDU Robotics

SDU

Topics to be covered in this course

→ Sampling and reconstruction
→ Aliasing
→ Quantization and dynamic range
→ Implementation
→ Conversion time-frequency domain
→ Z transform
→ Linear Time Invariant system (LTI)
→ System analysis
→ Window functions
→ Filter design
→ Impulse response (FIR and IIR)

SDU

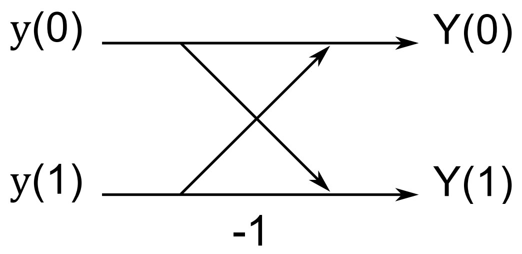
Computation complexity

We want to compute the N-Points DFT of $y(n)$, resulting $Y(m)$, where $m = 0, 1, \ldots, N-1$

$$Y(m) = \sum_{n=0}^{N-1} y(n)\, e^{-j\frac{2\pi}{N}mn}$$

N Points of DFT for the simplest model
N=2,

$$Y(m) = y(0) + y(1)e^{-j\pi m}, \qquad m = 0, 1$$
$$Y(0) = y(0) + y(1)$$
$$Y(1) = y(0) - y(1)e^{-j\pi}$$



Butterfly operation

The complexity for calculating the DFT equals to 2 times Addition

Suppose $N = 2^r$, $W_N = e^{-j\frac{2\pi}{N}}$

$$Y(m) = \sum_{n=0}^{N-1} y(n)\, e^{-j\frac{2\pi}{N}mn}$$

$$= y(0) + y(1)W_N^m + y(2)W_N^{2m} + y(3)W_N^{3m} + \ \dots + y(N-1)W_N^{(N-1)m}$$

$$= y(0) + y(2)W_N^{2m} + y(4)W_N^{4m} + \ \dots + \mathrm{y(N-2)}W_N^{(N-2)m} +$$
$$y(1)W_N^m + y(3)W_N^{3m} + y(5)W_N^{5m} + \cdots + y(N-1)W_N^{(N-1)m}$$

$$= y(0) + y(2)W_N^{2m} + y(4)W_N^{4m} + \ \dots + \mathrm{y(N-2)}W_N^{(N-2)m} +$$
$$W_N^m\left(y(1) + y(3)W_N^{2m} + y(5)W_N^{4m} + \ \dots + y(N-1)W_N^{(N-2)m}\right)$$

$$= \left(y(0) + y(2)W_M^m + y(4)W_M^{2m} + \ \dots + \mathrm{y(N-2)}W_M^{(M-1)m}\right) +$$
$$W_N^m\left(y(1) + y(3)W_M^m + y(5)W_M^{2m} + \ \dots + y(N-1)W_M^{(M-1)m}\right)$$

Let $M = N/2$

$W_N^2 = e^{-j\frac{2\pi}{N}*2}$
$= e^{-j\frac{2\pi}{N/2}} = W_{N/2}$
$= W_M$

$$Y(m) = Y_{even}(m) + W_N^m Y_{odd}(m)$$

$$Y(m) = \left(y(0) + y(2)W_M^m + y(4)W_M^{2m} + \ldots + y(N-2)W_M^{(M-1)m}\right) + W_N^m\left(y(1) + y(3)W_M^m + y(5)W_M^{2m} + \ldots + y(N-1)W_M^{(M-1)m}\right)$$

Why FFT is faster?

$\frac{N}{2}$ point DFT

$\frac{N}{2}$ point DFT

$$Y(m) = Y_{even}(m) + W_N^m Y_{odd}(m)$$

m from 0 to N-1

The complexity for calculating DFT_N equals to 2 times $DFT_{N/2}$ + N times Multiplication + N times Addition

Calculating $DFT_{N/2}$ requires 2 times $DFT_{N/4}$ + $\frac{N}{2}$ times Multiplication + $\frac{N}{2}$ times Addition

Therefore, overall, we need to calculate

$N/2$ times $DFT_{N/(N/2)}$ + $N * (\log_2 N - 1)$ times Multiplication + $N * (\log_2 N - 1)$ times Addition

It is about

$N * (\log_2 N - 1)$ times Multiplication + $N * \log_2 N$ times Addition

Calculate a DFT by direct use of the formulas, it requires N^2 times computation

When using the FFT algorithm, only $N * \log_2(N)$ times computation.

Matlab

Function:

- Fast Fourier Transform – fft()
- Shift zero-frequency to center – fftshift()


```
Fs = 1000;            % Sampling frequency
T = 1/Fs;             % Sampling period
N = 1500;             % Length of signal
t = (0:N-1)*T;        % Time vector
% construct a signal with a DC offset of amplitude 0.8, a 50 Hz sinusoid of amplitude 0.7, and a 120 Hz sinusoid of amplitude 1.
S = 0.8 + 0.7*sin(2*pi*50*t) + sin(2*pi*120*t);
% add random noise to the signal
X = S + 0.5*randn(size(t));

%plot to see the signal
figure();
plot(1000*t,X); hold on; plot(1000*t,S);
xlabel("t (milliseconds)")
ylabel("X(t)")

%% FFT
Y = fft(X);
f = Fs/N*(0:N-1);
% plot the complex magnitude using abs()
figure();
plot(f,abs(Y)/N,"LineWidth",3)    % remember to divide ‘N’
xlabel("f (Hz)")
ylabel("|fft(X)|")

%%
YY = fftshift(Y);
ff = Fs/N*(-N/2:N/2-1);
figure();
plot(ff,abs(YY)/N,"LineWidth",3)
xlabel("f (Hz)")
ylabel("|fft(X)|")
```

SDU

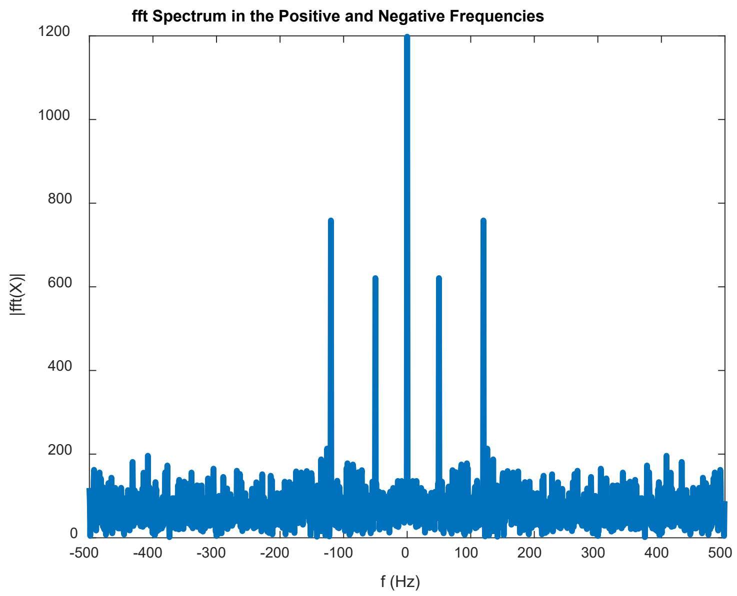
Matlab

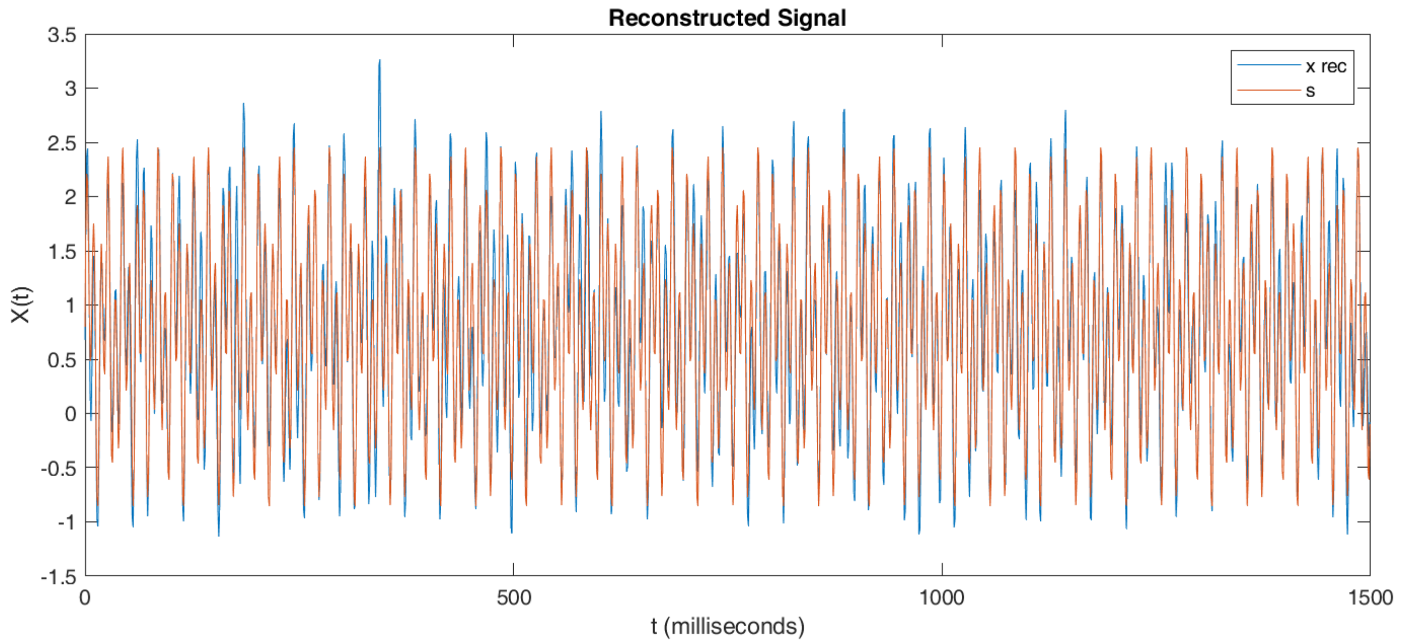
Function:

- Inverse zero-frequency shift – iffshift()
- Inverse Fast Fourier Transform – ifft()

```
% assume that I know frequency components above 150Hz are noise.
% remove noise by setting that range to 0
YY(ff>150)=0;
YY(ff<-150)=0;
plot(ff,abs(YY),"LineWidth",3)

%%
Y_rec = ifftshift(YY);
X_rec = ifft(Y_rec);
figure();
plot(1000*t,X_rec)
hold on
plot(1000*t,S)
title("Reconstructed Signal")
xlabel("t (milliseconds)")
ylabel("X(t)")
```





SDU

Exercise

→ 4.1
→ 4.2
→ 4.5
→ 4.21
→ 4.29